

好的，我把这份试卷里的题目（包括选择题、填空题、大题）全部按顺序做一遍，给出答案和简要解析。

一、选择题

第1题

用 8 位有符号数表示的十进制数 -43，扩展成 16 位有符号数，其编码为（ ）

-43 的 8 位二进制补码：1101 0101 (0xD5)

符号扩展成 16 位：1111 1111 1101 0101 = 0xFFD5

答案：B. 0xFFD5

第2题

十进制数 -6 的单精度浮点数 IEEE754 代码为（ ）

$-6 = -110.0_2 = -1.1_2 \times 2^2$

符号位 S = 1

阶码 E = 2 + 127 = 129 = 1000 0001

尾数 = 100000000000000000000000 (去掉隐含的1)

组合：1 10000001 100000000000000000000000

= 1100 0000 1100 0000 0000 0000 0000 0000

= 11000000110000000000000000000000

答案：A. 11000000110000000000000000000000

第3题

缓存：块大小64字节，8路组相联，总容量32KB，地址 0x601018 命中，求组号

- 块大小 64B $\rightarrow$ 块内偏移 = $\log_2 64 = 6$ 位
- 每组块数 = 8 路
- 总块数 = 32KB / 64B = 512 块
- 组数 = 512 / 8 = 64 组 $\rightarrow$ 组索引 = $\log_2 64 = 6$ 位

地址 0x601018 = 0110 0000 0001 0000 0001 1000 (32位)

低6位偏移：01 1000 = 0x18

接下来6位组索引：0000 00 = 0x00

答案：C. 0x00

第4题

将 sum.c 编译为汇编代码的命令

- -S 生成汇编代码 (.s 文件)

答案：C. gcc -S sum.c

第5题

指令：movb 70,80,%bl; addb %al,%bl

70 + 80 = 150 = 0x96 (8位溢出，保留低8位)

答案：A. 0x96

第6题

答案：

解析

这道题考查的是链接符号 (linker symbol) 的概念，即哪些名字会成为全局符号进入符号表。

先看代码中的每个标识符：

```
int data[100];           // 全局变量 → 符号 data
int main() {             // 函数 → 符号 main
    static void * handle; // static 全局变量（文件作用域）→ 符号 handle
    void (* addvec)(...); // 局部变量（函数内定义）→ 不是链接符号
    handle = dlopen(...);
    addvec = dlsym(handle, "addvec");
    addvec(x, y, z, 2);
    return 0;
}
```

逐个选项分析：

选项	标识符	是否为链接符号?	原因
A. data	data	☑ 是符号	全局数组，在 .data 段，编译器会生成全局符号
B. main	main	☑ 是符号	函数名，全局符号，在 .text 段
C. handle	handle	☑ 是符号	static 修饰的全局变量，文件作用域，虽然是局部符号 (LOCAL)，但仍然是符号表里的符号
D. addvec	addvec	✗ 不是符号	它只是 main 函数内部的局部变量（函数指针），不会进入符号表

关键知识点

1. 什么情况下标识符会成为符号？

- **全局变量**（包括 `static` 全局变量）→ 符号
- **函数名** → 符号
- **局部变量**（包括函数内的指针变量）→ **不是符号**（只存在于栈上）

2. `static` 的作用

- `static void * handle;` 是**文件作用域的静态全局变量**，它仍然是符号，只是符号的绑定属性是 `LOCAL`（仅本文件可见）

3. `dlsym` 中的 "addvec"

- 这是**字符串常量**，不是变量名，不产生符号

题目问的是“哪个选项不是代码中的符号？”

D. `addvec` 是局部变量，不是链接符号。

所以**正确答案应该是 D**，

总结

- **符号 (Symbol)**：全局变量、函数名、静态全局变量
- **非符号**：局部变量（包括函数指针）、形参、字符串常量

所以这道题选 **D. `addvec`**。

第7题

关于 **SRAM** 和 **DRAM**，不正确的是（ ）

- A. 正确，都易失
- B. **错误**，DRAM 比 SRAM 慢
- C. 正确，SRAM 成本高
- D. 正确，DRAM 需要刷新

答案：B

第8题

答案：C

解析

逐项分析：

选项	描述	是否正确	原因
A	栈底位于高地址，栈顶位于低地址	☑ 正确	x86-64 栈向低地址增长，栈顶（%rsp）地址最小
B	函数执行完毕，栈帧会被销毁	☑ 正确	函数返回时恢复 %rsp 和 %rbp，栈帧逻辑上销毁
C	函数中声明的静态局部变量会保存在对应的栈帧中	✗ 错误	静态局部变量 （ <code>static</code> ）存储在 .data 或 .bss 段中， 不在栈上 ，生命周期为整个程序运行期间
D	如果函数的参数超过 6 个，则超出的部分会使用栈存储	☑ 正确	x86-64 System V ABI 规定：前 6 个参数用寄存器（%rdi, %rsi, %rdx, %rcx, %r8, %r9），第 7 个及以后用栈传递

关键知识点补充

1. 栈帧中存放的内容

- 局部变量（非 `static`）
- 被保存的寄存器值
- 返回地址
- 超过 6 个的函数参数（调用者栈帧中）

2. 静态局部变量的存储

```
void func() {  
    static int x = 0;    // 不在栈上，在 .data 段  
    int y = 0;           // 在栈上  
}
```

- `x` 在程序加载时分配，函数返回后仍然存在
- 多次调用 `func()`，`x` 保持上次的值

3. x86-64 参数传递规则（System V ABI）

参数序号	存储位置
1~6	寄存器：%rdi, %rsi, %rdx, %rcx, %r8, %r9
7+	栈（从右到左压栈）

所以答案是 C。

第9题

存储器速度排序正确的是（）

寄存器 > 高速缓存 > 内存 > 固态硬盘 > 磁盘

- A. 错（Cache 比内存快）
- B. 错（磁盘比 SSD 慢，内存比磁盘快）
- C. 错（固态硬盘比内存慢）
- D. 正确（寄存器 > 内存 > 磁盘）

答案：D

第10题

x86-64 控制流分支描述正确的是（）

- A. 错，还有无条件跳转等
- B. 正确，条件码可通过 setX 等访问
- C. 错，条件数据传输不涉及跳转
- D. 错，分支计算量大时不适合条件数据传输

答案：B

第11题

存储器系统层次结构说法正确的是（）

- A. 错，越往下速度越慢，不是越快
- B. 错，主存不会超过 Cache
- C. 正确，Cache 目的是提高等效访问速度
- D. 错，磁盘还解决容量和非易失性

答案：C

第12题

汇编代码对应的 C 语言

汇编：

```
testq %rsi, %rsi
je .L1
cmpq %rdi, (%rsi)
jge .L1
movq %rdi, (%rsi)
.L1 ret
```

对应逻辑：

如果 p 不为空，且 $*p < a$ ，则 $*p = a$

答案：B

二、填空题

第1题

十进制转二进制（8位）转十六进制（2位），已知 $0xCE = 1100\ 1110 = 206$

十进制	二进制（8位）	十六进制（2位）
206	1100 1110	0xCE

第2题

```
char c = -0x2A
```

$0x2A = 42$ ， -42 的补码：

$42 = 0010\ 1010$ ，取反加1 = $1101\ 0110 = 0xD6$

答案：0xD6

第3题

$0x12345678$ 小端存储在 $0x1000$ 起始

小端：低地址存低字节

地址 $0x1000 \rightarrow 0x78$

$0x1001 \rightarrow 0x56$

答案：0x56

第4题

-94 的 8 位补码

$94 = 0101\ 1110$ ，取反加1 = $1010\ 0010$

答案：10100010

第5题

```
short s = 0xABCD, char c = s
```

short 转 char，截断低字节：0xCD

存储在存储器中就是 0xCD

答案：0xCD

第6题

```
addl $1, (%rdi, %rax, 4)
```

基址 %rdi，索引 %rax，比例 4（int 大小）

所以 %rdi 存放数组首地址，%rax 存放下标

答案：数组首地址（基址），下标

第7题

磁盘厂商 1TB = 10^{12} = 1,000,000,000,000 字节
操作系统 1TiB = 2^{40} = 1,099,511,627,776 字节

答案：10¹²，2⁴⁰

第8题

内存通常用 DRAM 实现，高速缓存用 SRAM

答案：DRAM，SRAM

第9题

汇编是统计二进制中 1 的个数 (popcount)

缺失语句：

- ① `result = 0;`
- ② `x & 1` (或 `x & 0x1`)
- ③ 函数功能：统计 x 的二进制表示中 1 的个数

答案：

- ① `result = 0`
 - ② `x & 1` (或 `x & 0x1`)
 - ③ 统计 x 中 1 的位数
-

第10题

已知 %rax = 0x100, %rcx = 0x4

内存数据（小端，每格1字节）：

地址	0	1	2	3	4	5	6	7
0x100	06	03	00	00	0C	08	00	00
0x108	FF	FF	FF	FF	CC	CC	CC	CC
0x110	00	00	00	00	04	00	00	00
0x118	55	55	00	00	21	20	00	00

指令	结果 (16进制)
<code>movq %rax, %rdx</code>	<code>0x100</code>
<code>movq (%rax), %rdx</code>	<code>0x0000080c00000306</code>
<code>movw 12(%rax), %dx</code>	<code>0xC000</code>
<code>movq (%rax, %rcx, 4), %rdx</code>	<code>0x0000000040000000</code>
<code>movl 16(%rax, %rcx), %edx</code>	<code>0x00000004</code>
<code>leaq 8(%rax, %rcx, 2), %rdx</code>	<code>0x110</code>

三、阅读以下 C 代码，回答问题

(1) 强符号

文件	强符号
<code>m1.c</code>	<code>x</code> (已初始化全局变量)、 <code>y</code> (已初始化全局变量)、 <code>p1</code> (函数定义)
<code>m2.c</code>	<code>main</code> (函数定义)

(2) 弱符号

文件	弱符号
<code>m1.c</code>	无
<code>m2.c</code>	<code>x</code> (未初始化全局变量)

四、在 x86-64 的 Linux 系统中声明如下的结构体 sRec，请回答下列问题

好的，我们逐步分析这个结构体的内存对齐问题。

一、基本规则 (x86-64 System V ABI)

在 x86-64 Linux 系统中：

数据类型	大小 (字节)	对齐要求 (地址必须是其倍数)
<code>short</code>	2	2
<code>int</code>	4	4
<code>int *</code> (指针)	8	8
<code>char</code>	1	1
结构体整体对齐	成员中最大对齐要求	8 (因为指针是8)

对齐原则：

- 1. 每个成员的偏移量必须是其**对齐要求的倍数**
- 2. 结构体总大小必须是**最大对齐要求的倍数**（末尾填充）

二、原结构体分析

```
struct {
    short a;           // 2字节，对齐2
    int b[2];          // 8字节（2个int），对齐4
    int *next;         // 8字节，对齐8
    char c;            // 1字节，对齐1
} sRec;
```

(1) 各字段偏移量计算

字段	大小	对齐	起始偏移	占用字节	说明
a	2	2	0	0~1	从0开始，对齐2满足
填充	-	-	2~3	2字节	为了让 b 从4开始（对齐4）
b[0]	4	4	4	4~7	从4开始，对齐4满足
b[1]	4	4	8	8~11	紧接前一个
填充	-	-	12~15	4字节	为了让 next 从16开始（对齐8）
next	8	8	16	16~23	从16开始，对齐8满足
c	1	1	24	24	紧接前一个
末尾填充	-	-	25~31	7字节	为了让总大小是8的倍数（最大对齐8）

各字段偏移量答案：

- a 偏移 = 0
- b[0] 偏移 = 4（b 数组整体起始偏移为4）
- b[1] 偏移 = 8
- next 偏移 = 16
- c 偏移 = 24

(2) 整个结构体大小

从上面计算可知，总大小为 **32 字节**。

答案：32 字节

三、重新排列使浪费空间最小

思路：按对齐要求从大到小排列，减少填充。

对齐要求排序：指针（8）> int（4）> short（2）> char（1）

推荐排列

```
struct {
    int *next;      // 8字节，对齐8，偏移0
    int b[2];       // 8字节，对齐4，偏移8
    short a;        // 2字节，对齐2，偏移16
    char c;         // 1字节，对齐1，偏移18
} sRec;
```

(3) 新结构体各字段偏移量

字段	大小	对齐	起始偏移	占用字节	说明
next	8	8	0	0~7	从0开始
b[0]	4	4	8	8~11	从8开始（8是4的倍数）
b[1]	4	4	12	12~15	紧接前一个
a	2	2	16	16~17	从16开始（16是2的倍数）
c	1	1	18	18	紧接前一个
末尾填充	-	-	19~23	5字节	为了让总大小是8的倍数（最大对齐8）

新各字段偏移量：

- next 偏移 = 0
- b[0] 偏移 = 8
- b[1] 偏移 = 12
- a 偏移 = 16
- c 偏移 = 18

新结构体总大小 = 24 字节

四、对比总结

项目	原结构体	优化后结构体
各字段偏移	a=0, b=4, next=16, c=24	next=0, b=8, a=16, c=18
总大小	32 字节	24 字节
节省空间	-	节省 8 字节 (25%)

五、验证：另一种排列方式

如果按 `int b[2]`（8字节，对齐4）放在最前面：

```
struct {
    int b[2];           // 8字节，对齐4，偏移0
    int *next;          // 8字节，对齐8，偏移8（8是8的倍数，OK）
    short a;            // 2字节，对齐2，偏移16
    char c;             // 1字节，对齐1，偏移18
} sRec;
```

总大小也是 **24 字节**，因为：

- 最大对齐还是8
- 末尾填充到24（18→24，补5字节）

所以这个排列也正确，大小同样是24字节。

六、核心知识点

1. **偏移量计算**：起始地址必须是对齐要求的整数倍
2. **结构体总大小**：必须是最大对齐要求的整数倍（末尾填充）
3. **优化技巧**：按对齐要求从大到小排列字段，减少填充浪费

如果有任何不清楚的地方，可以继续问我。

五、在某具有三级流水线的处理器中，每级流水线的组合逻辑和寄存器的延迟时间如下图所示(1s = 10¹²ps)，请回答

A	寄存器	B	寄存器	C	寄存器
100	20	100	20	100	20

情况	最慢级延迟	时钟周期	吞吐量
原始（B=100ps）	120 ps	120 ps	8.33 GIPS
变慢后（B=120ps）	140 ps	140 ps	7.14 GIPS

六、转置缓存题（已在前面详细回答）

好的，这道题是经典的**矩阵转置缓存命中率分析**，我们来完整解答。

一、问题分析

已知条件

- 矩阵 `a[N][N]` 和 `b[N][N]`，元素类型为 `int`（4 字节）
- 缓存块大小 = 16 字节 = **4 个 int**
- N 非常大（远大于缓存能容纳的行数）
- 缓存总容量不足以同时缓存多行数据

核心代码

```
for (i = 0; i < N; i++) {  
    for (j = 0; j < N; j++) {  
        b[j][i] = a[i][j];  
    }  
}
```

二、访问模式分析

1. 访问 `a[i][j]`（读操作）

- `i` 固定, `j` 递增 → **按行连续访问**
- 每次连续读取 4 个 int（刚好一个缓存块）
- 每个缓存块只有**第一次访问未命中**（冷启动），后续 3 次命中
- **未命中率 $\approx 1/4$**

2. 访问 `b[j][i]`（写操作）

- `j` 变化, `i` 固定 → **按列访问**
- 每次访问的地址间隔为 `N * 4` 字节（一行的大小）
- 因为 `N` 很大且缓存不足以缓存多行，每次访问几乎都在不同的缓存行
- **未命中率 ≈ 1** （几乎每次都未命中）

三、（1）平均每次内循环的未命中次数

每次内循环迭代访问两个元素：

- `a[i][j]`：贡献 **$1/4$** 次未命中
- `b[j][i]`：贡献 **1** 次未命中

平均每次内循环未命中次数 = $1/4 + 1 = 1.25$ 次

四、（2）总的未命中次数

内循环总共执行 `N × N` 次迭代。

总未命中次数 = $1.25 \times N^2 = 5N^2/4$

五、（3）优化方案

核心问题

`b[j][i]` 的按列访问破坏了空间局部性，导致每次写操作都未命中。

优化思路：分块（Blocking / Tiling）

把大矩阵分成小块，每次处理一个块内的数据，让数据在块内连续访问，充分利用缓存。

优化代码

```
int transpose(int a[N][N], int b[N][N]) {
    int i, j, ii, jj;
    int B = 4; // 块大小，可根据缓存调整

    for (ii = 0; ii < N; ii += B) {
        for (jj = 0; jj < N; jj += B) {
            for (i = ii; i < ii + B && i < N; i++) {
                for (j = jj; j < jj + B && j < N; j++) {
                    b[j][i] = a[i][j];
                }
            }
        }
    }
    return 0;
}
```

为什么有效？

优化前	优化后
内层循环跨越整个矩阵行	内层循环限制在 B×B 小范围内
b 按列跳跃访问（跨 N 行）	b 的访问在块内连续
几乎每次 b 访问都未命中	b 的未命中率也降到约 1/4
总未命中 $\approx 1.25N^2$	总未命中 $\approx 0.5N^2$ （约降低 60%）

块大小选择原则

- **B 不能太大**：B×B 的数据必须能放进缓存
- **B 不能太小**：太小会增加循环开销
- 常用值：4、8、16（根据缓存大小调整）

六、最终答案汇总

问题	答案
(1) 平均每次内循环未命中次数	1.25 次
(2) 总未命中次数	1.25N ² （即 5N ² /4）
(3) 优化方案	分块（Blocking），块内连续访问，代码如上

七、补充：计算优化后的未命中次数（参考）

假设块大小 $B = 4$ ，每个块完全在缓存中：

- 每个块内： a 访问 B^2 次，未命中 $B^2/4$ 次
- b 访问 B^2 次，未命中 $B^2/4$ 次
- 每个块总未命中 $\approx B^2/2$
- 块总数 $= (N/B)^2 = N^2/B^2$
- 总未命中 $\approx (N^2/B^2) \times (B^2/2) = N^2/2$

优化前： $1.25N^2$ ，优化后： $0.5N^2$ ，减少了 60% 的未命中。

如果需要更详细的推导或不同块大小的对比，可以继续问我。

七、阅读以下代码，回答问题。

问题	答案
(1) 数据空间局部性	差。因为最内层遍历 k ，地址跳跃 $4N^2$ 字节，不连续
(2) 指令时间局部性	好。循环体指令反复执行，保留在指令缓存中
(3) 优化方案	调整循环顺序为 $k \rightarrow i \rightarrow j$ ，让最内层遍历最后一个维度 j ，实现连续内存访问

八、请简述x86-64系统中的过程调用规范。详细说明过程调用机制中的控制流转移，数据传输和过程中数据存储管理的具体实现方式。

过程调用规范：假设过程 P 调用过程 Q ， Q 执行后返回到 P 。这些动作包括以下机制：传递控制、传递数据、分配和释放内存。

控制流转移：将控制从程序 P 转移到函数 Q ，需要用指令 `call Q` 把返回地址压入栈中，并将程序计数器 PC 设置为 Q 的起始位置；结束后 `ret` 指令会从栈中弹出返回地址，并将程序计数器 PC 设置为弹出的返回地址。

数据传输：通过寄存器最多传递 6 个整型参数，超出 6 个的部分通过栈来传递，且这些参数存储在调用者 P 的栈帧中。

过程中数据存储管理：当过程中需要的存储空间超过寄存器能够存放的大小时，就会在栈上分配空间。这个部分称为过程的栈帧。当前正在执行过程的帧总是在栈顶。寄存器分为调用者保存寄存器和被调用者保存寄存器。

九、/home/user 目录下存在一个文件 main.c，需要将 main.c 文件复制到/home/user/mylab 目录下（该目录当前不存在），应该执行哪些命令才能完成上述工作（当前工作目录为/home/user）

```
mkdir -p mylab
cp main.c mylab/
```

十、假设有一个具有如下属性的系统：

内存是字节寻址的

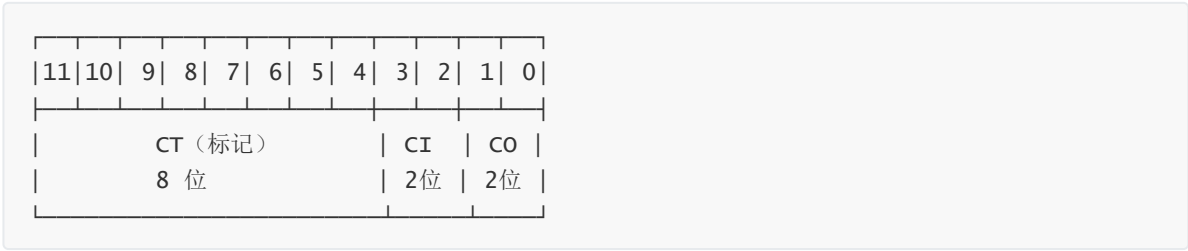
每次访问内存都是 1 字节的

地址位宽为 12

高速缓存是组相联的，有 4 个组，每个组 2 个块，块大小为 4 字节

高速缓存的内容如下，所有地址，标记，值都以 16 进制表示。

(1) 下图给出了物理地址的格式（每个小框表示一位），请在图中标出用来确定下列信息所对应的位：CO 高速缓存块内偏移，CI 高速缓存组索引，CT 高速缓存标记。



(2) 对于下列内存访问，当他们按照顺序依次执行时，请指出高速缓存是否命中，如果可以推断出访问的信息，请给出值

操作	地址	命中 (Y/N)	读出的值 (或未知)
读	0x831	Y	0x97
写	0x833	Y	未知 (N/A)